

CMSC 104 Section 02

Fall 2026

Project #1: Math Operations with and Without the Python math() library

Date Assigned: September 9, 2026

Date Due: September 23, 2026

Assignment:

Part 1: RSA encryption algorithm

The RSA (Rivest-Shamir-Adelman) encryption algorithm has underlain the security of the Internet for much of the last two decades. It is a public-key or asymmetric algorithm. Every user has two related keys: an encryption key, e , that is public and is used to encrypt any private messages sent to that user; and a decryption key, d , that is secret and is only known to the user.

The other important part of the algorithm is a modulus, m . All math is modular math; all numbers are stated modulo m .

To encrypt plaintext t and get ciphertext c , you calculate

$$c = (t^e) \% m$$

And to decrypt c and get back the plaintext, p , you calculate

$$t = (c^d) \% m$$

All values to be encrypted are treated as integers, and they must be smaller than m . If you are encrypting something other than an integer, convert it to an integer (the details are not part of this assignment). If the integer you get is larger than m , split it into pieces (again, the details are not part of this assignment).

The two keys e and d are related to each other in that they are multiplicative inverses modulo r , where r is related to m (see below). That is,

$$(e \cdot d) \% r = 1.$$

The modulus m is the product of two chosen prime numbers, p and q . Calculate

$$m = pq$$

So, your program should:

1. Print out a welcoming message telling the user what the program will do.
2. Prompt the user to enter the two prime numbers, p and q . The user will enter the numbers above. Remember that the values will be read in as strings; you'll have to convert them to integers
3. Calculate m as shown above. Also calculate $r = (p-1)(q-1)$
4. Prompt the user to enter the public key. Again, this will be read in as a string. Don't forget to convert.
5. Calculate the secret decryption key, d . To do this use insert the following statement in your code:

```
d = pow(e, -1, r)
```

“pow” is a predefined Python function that returns $e^{-1} \bmod r$. It does the calculations for you.

6. Prompt the user to enter the secret message- the plaintext, t .. Tell the user that this must be an integer less than m . You do not need to do error checking in Project 1 - the user will enter a valid number.
7. Calculate the ciphertext, c , as described above - $c = (t**e)\%m$, and print it out.
8. Then calculate the plaintext, t , as described - $t = (c**d)/m$. Print it out to show the user that the calculated value is exactly what was entered.

Part 2: Use of the math() library

Write a Python program that does the following, in the order specified:

1. Imports the Python math library.
2. Prints out a message to welcome the user to the program, and tells the user what the program will do.
3. Ask the user to enter a positive integer. NOTE: you need not do error checking in project 1. You can assume that the user will actually enter a positive integer.
4. Calculate the sum of all integers between 1 and the user's number, inclusive. Print out the original number and the sum, clearly labeled as to what each number means.
5. Calculate the square root of the sum you obtained in step 4, using the math library. Print this out, clearly labeled as to what it is.
6. Calculate the factorial of the number. That is, the product of all the integers between 1 and the user's number, inclusive. Print out the original number and the factorial, clearly labeled as to what each number means.
7. Calculate the cube root of the factorial you obtained in step 6, clearly labeled as to what it is.
8. Print out a message to the user thanking the user for using your program, and then terminate.